



Optimized autonomous navigation for field robots: extended results and practical deployment

J. Rakun¹ · G. Popič¹

Received: 12 September 2025 / Accepted: 17 November 2025 / Published online: 9 December 2025
© The Author(s) 2025

Abstract

Purpose This study introduces an optimized algorithm for autonomous navigation of field robots, aiming to improve navigation accuracy, reduce crop damage and shorten execution times in agricultural environments.

Methods The enhanced solution integrates advanced data filtering with sensor fusion techniques, combining LiDAR and IMU inputs to produce precise 3D point cloud representations for reliable navigation in structured crop rows. Both the legacy and improved algorithms were evaluated through simulation and physical trials on the FarmBeast robotic platform.

Results The improved algorithm reduced traversal time by up to 33% on certain field sections and lowered crop damage by 25% compared to the previous version.

Conclusions Results confirm the robustness and effectiveness of the enhanced navigation system in complex agricultural field conditions, demonstrating its potential for practical deployment within farming automation.

Keywords Autonomous operation · Agricultural automation · Sensors · State machine · ROS

Introduction

The automation of agricultural field tasks has gained increasing importance with the rise of precision farming, where accuracy and efficiency are essential (Raj et al., 2024). Autonomous navigation in structured crop environments, such as regularly spaced maize rows, introduces both challenges and opportunities, particularly when robots must minimize crop damage while maintaining high operational speed (Panda et al., 2023). The FarmBeast project, launched as a student collaboration, addresses these challenges by developing and

✉ J. Rakun
juri.rakun@um.si

¹ Faculty of Agriculture and Life Sciences, University of Maribor, Pivola 10, Hoče, Slovenia

refining an autonomous agricultural robot designed for field navigation and participation in events such as the Field Robot Event (Bernad et al., 2019).

Autonomous navigation in agricultural robotics has advanced significantly through multi-sensor fusion, intelligent perception, and adaptive path-planning approaches. Multi-sensor fusion remains a cornerstone of robust field navigation, enabling robots to operate reliably in dynamic and semi-structured environments. Rakun et al. (2022) demonstrated a practical sensor fusion-based localization framework for the Rovitis 4.0 vineyard robot, combining odometry, IMU, and RTK GPS data via a sensor-specific state machine to achieve centimeter-level positioning accuracy under occluded and vibration-prone field conditions. Similarly, Lv et al. (2022) proposed a loosely coupled Extended Kalman Filter for agricultural scene-based multi-sensor fusion, enhancing robustness when individual sensors fail or degrade. In a comparable line of research, Li et al. (2025) presented an octree-based LiDAR RRT navigation method for orchard robots, focusing on computational efficiency and high-precision path following rather than sensor fusion integration.

In parallel, path-planning research has evolved from deterministic and heuristic algorithms toward adaptive hybrid and learning-based approaches. Yahaya et al. (2022) provided a comprehensive taxonomy of agricultural robot path-planning methods, emphasizing hybrid strategies that integrate classical graph search with bio-inspired optimization to enhance adaptability in irregular terrain. At the same time, deep reinforcement learning (DRL) has emerged as a promising framework for real-time, model-free navigation. Baby et al. (2024) successfully applied DRL to a four-wheel-steer field robot for crop-row tracking and adaptive maneuvering, achieving robust navigation without prior maps. Collectively, these studies highlight a rapidly advancing research landscape where sensor fusion, adaptive path planning, and learning-based control jointly drive progress toward precise, reliable, and efficient autonomous navigation in agricultural environments.

Within this context, the FarmBeast project, launched as a student collaboration, addresses these challenges by developing and refining an autonomous agricultural robot designed for field navigation and participation in events such as the Field Robot Event (Bernad et al., 2019). The Field Robot Event (2024) emphasizes navigation through structured crop rows, such as regularly spaced maize rows, without causing plant damage, making reliable navigation a central task (Rakun, 2015). Through its annual development cycle, FarmBeast has progressively incorporated advanced technologies, including sensor upgrades and algorithmic improvements, to meet these demanding requirements (FarmBeast, 2024). In its latest version, the platform integrates a Velodyne VLP-16 LiDAR (Velodyne, 2015) and an Xsens MTi-610 Inertial Measurement Unit (IMU) (Xsens, 2015), which together enable high-precision 3D point cloud mapping of the environment. These sensor inputs are further processed using advanced filtering and fusion techniques, ensuring robust navigation performance even under challenging field conditions (Popič et al., 2024).

The main objective of this study is to develop and evaluate an optimized autonomous navigation algorithm for the FarmBeast robotic platform. The work specifically aims to improve upon the previously developed legacy approach (Popič, 2021) by enhancing positioning accuracy, increasing traversal speed, and reducing crop damage. To achieve this, the optimized algorithm is systematically compared against the legacy system through both simulation and controlled physical trials. In doing so, the study demonstrates how targeted optimization of perception and control strategies can significantly improve the reliability

and efficiency of autonomous field robots, underscoring their potential role in advancing precision agriculture and sustainable farming practices.

Materials and methods

This section outlines the hardware architecture, sensor fusion methods, and simulation tools employed to optimize the FarmBeast's autonomous navigation, as well as the data filtering and control algorithms that enable precise operation in complex agricultural environments.

Hardware and Computational Setup – FarmBeast is a modular agricultural robot platform designed for adaptability across different tasks. Its computational framework combines a Raspberry Pi and an Intel NUC. The Raspberry Pi manages actuator control, while the Intel NUC—equipped with an Intel i7 processor—handles computationally demanding processes such as sensor data integration and navigation algorithms. Figure 1 shows the latest version of the FarmBeast robot, with both its mechanical design and practical implementation.

Sensors – The FarmBeast platform is equipped with a Velodyne VLP-16 LiDAR and an Xsens MTI-610 IMU. The LiDAR captures 3D representations of the surrounding environment, producing a dense point cloud with a 360° horizontal field of view and a 30° vertical range, essential for accurate mapping and obstacle detection. The IMU provides critical orientation and acceleration data, utilizing quaternion-based orientation tracking (Zhang et al., 2020) to overcome the discontinuity limitations of Euler angles, thereby improving navigation robustness in agricultural conditions. This distinctive feature of the proposed algorithm replaces traditional Euler angles with quaternions to enhance robustness and reliability during turns.

Comparative system overview – The FarmBeast navigation system builds on a legacy framework (Popič, 2021) that provided the first implementation of LiDAR, IMU and odometry-based navigation in structured crop rows. Several key elements of that system were retained in this work, including the hardware platform (Intel NUC and Raspberry Pi), the primary sensor suite (Velodyne LiDAR and Xsens IMU), and the general principle of using crop row detection to guide the robot along a centered trajectory.

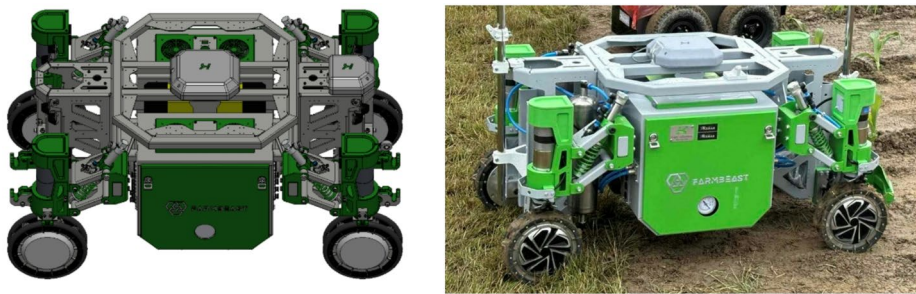


Fig. 1 The FarmBeast robotic platform. Left: CAD model showing the mechanical design with sensor and actuator placement. Right: photograph of the implemented robot used in experiments. The platform integrates a Velodyne VLP-16 LiDAR on top and an Xsens MTI-610 IMU inside the chassis, supported by an Intel NUC for computation and a Raspberry Pi for actuator control

In contrast, the present study introduces several optimized components that form the core contributions of this work: (a) **A multi-stage filtering pipeline** (voxel grid reduction, RANSAC ground-plane removal, trapezoid polygon filtering) to reduce computational load while preserving navigation-critical features. (b) **Quaternion-based orientation tracking**, which improves robustness and reliability during turning compared to the legacy use of Euler angles. (c) **A modular finite-state machine (FSM)**, which enables smoother and more adaptable transitions between navigation states such as row following, turning, and obstacle avoidance.

Legacy approach to navigation – The navigation system developed by Popič (2021) formed the basis for comparison with the optimized algorithm presented here. It was originally designed and tested in Robot Operating System (ROS, n. d.) and Gazebo (Open Robotics, n. d.), focusing on autonomous driving of a simulated agricultural robot in structured row environments. The algorithm utilized data from LiDAR, IMU and odometry to maintain robust navigation in fields with partial irregularities.

Row detection was performed using LiDAR measurements. Raw point clouds were restricted to a forward-looking region to reduce disturbances from leaves protruding into the inter-row space. By averaging distances from the left and right crop rows, the algorithm adjusted steering dynamically to keep the robot centered. For curved rows, deviations in LiDAR readings triggered proportional angular corrections and speed adjustments, reducing forward velocity when sharper turns were required.

At the end of a row, navigation switched to IMU-based control. When LiDAR no longer detected plants, the robot recorded its yaw orientation and executed a 180° turn to align with the next row. The direction of the initial turn (left or right) was set via a configurable launch parameter, enabling adaptation to different field layouts.

Data Filtering – To manage the large volume of LiDAR data, a multi-stage filtering pipeline is applied. A voxel grid filter (Safe Software, n.d.) first reduces data density by grouping points into spatial cells, lowering computational load while preserving structural detail. Subsequently, a RANSAC (Random Sample Consensus) algorithm (Fischler & Bolles, 1981) identifies and removes ground points, isolating relevant data for navigation. This combined approach increases efficiency by focusing on essential features. Figure 2 illustrates the difference between unfiltered and filtered data.

To further streamline processing, additional filtering was implemented to discard irrelevant data, keeping only points corresponding to crop rows near the robot. Several poly-

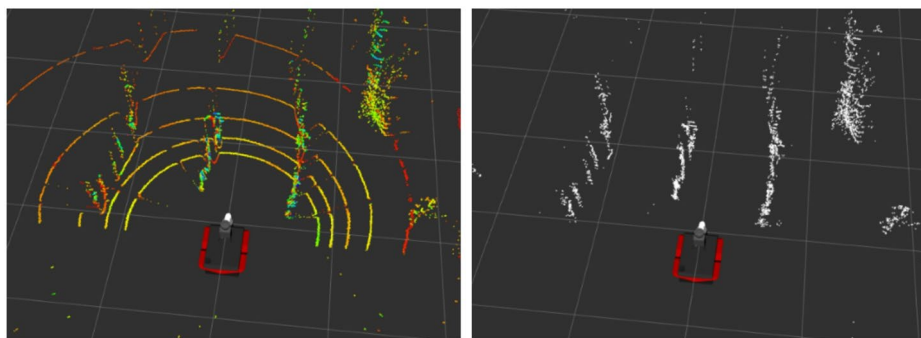


Fig. 2 Comparison of LiDAR point clouds before (left) and after (right) the filtering pipeline, displayed in the ROS Rviz environment. Each plot shows a plan (x–y) view of the robot's surroundings

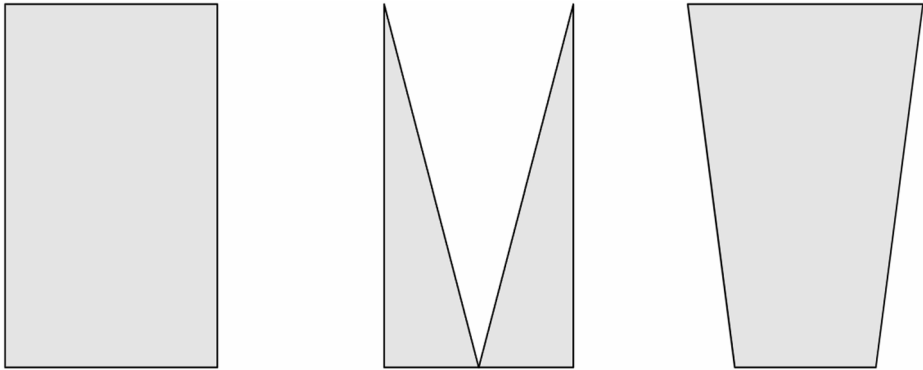


Fig. 3 Overview of different polygonal filtering shapes applied to LiDAR point clouds. Retained voxels (crop-row points) are highlighted, while points outside the polygon are discarded. This process reduces computational load by focusing only on relevant regions near the robot

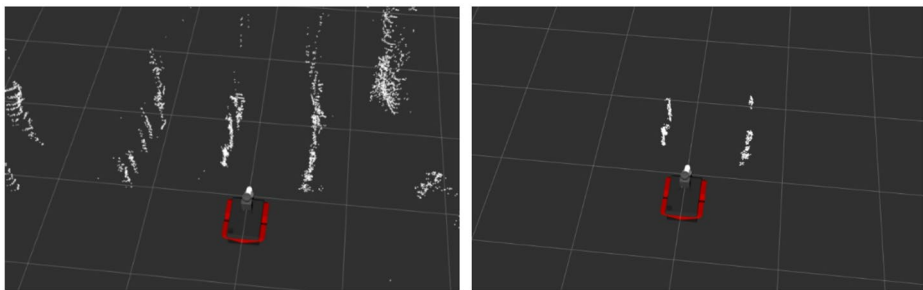


Fig. 4 LiDAR point cloud data at two stages of the filtering process. Left: intermediate filtering after voxel grid and RANSAC ground plane removal. Right: final result after applying the trapezoid-shaped polygon filter, retaining only crop-row points for navigation

gon-based approaches were tested, with the trapezoid-shaped polygon yielding the most effective results, and thus adopted for all subsequent data processing. Figures 3 and 4 illustrate the polygon design and the resulting filtered dataset.

Control architecture State Machine Implementation – The navigation logic is governed by a finite-state machine (FSM) implemented in ROS using the SMACH library (Bohren, n.d.). This structure enables flexible control across different navigation states, including row following, turning, and obstacle avoidance, while dynamically adapting to changing field conditions. The modular FSM design not only improves robustness but also facilitates the integration of new functionalities for future upgrades.

Figure 5 shows an example of the FSM implementation used for advanced navigation tasks at the Field Robot Event (FRE). The sequence begins with initialization, followed by row navigation until the row is completed. The robot then performs a two-phase turn: first a 90° rotation, then a forward movement based on odometry, and finally another 90° turn into the next row.

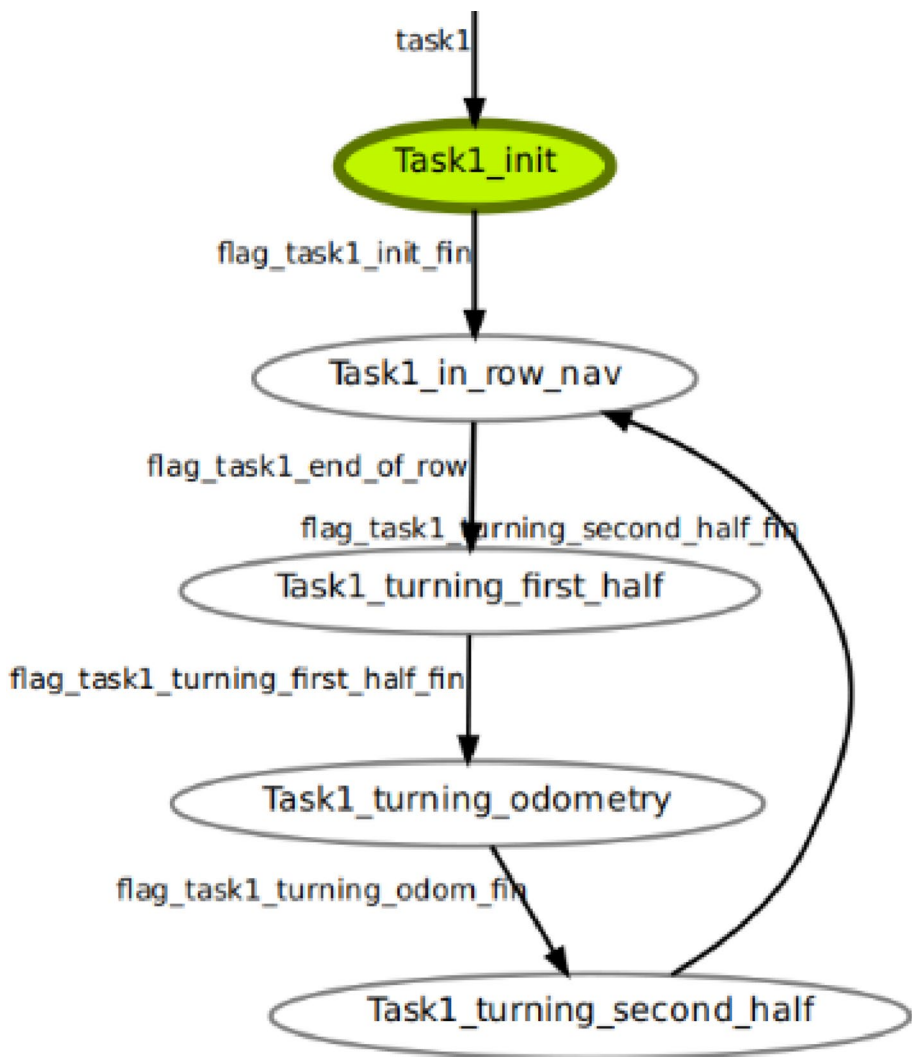


Fig. 5 Finite-state machine (FSM) design for FarmBeast navigation, implemented in ROS SMACH. The diagram illustrates transitions between states: initialization, row following, turning, and realignment. This modular control structure enables flexible adaptation to different field layouts and supports advanced navigation tasks used at the Field Robot Event

Simulation Environment – Development and validation of the algorithm were carried out using Gazebo and Rviz (Willmann & Krajnik, 2015), two key tools in ROS-based robotics. Gazebo provides a physics-based environment capable of replicating realistic agricultural conditions such as soil friction and obstacle interactions. Rviz complements this with real-time visualization of sensor data and robot trajectories, allowing for efficient testing and fine-tuning of the navigation algorithm before field deployment. Together, these tools significantly reduced development risk while enhancing system robustness.

Navigation algorithm

Navigation represents a core component of autonomous mobile robot operation, defining both the movement direction and control strategies required to achieve reliable field performance. The input to the navigation algorithm consists of data obtained after filtering and limiting the sensor field of view. For reference, the start of the coordinate system (0,0) is defined at the beginning of the field, corresponding to the robot's initial position. This data is then divided into two subsets: points located to the left and points located to the right of the robot. Each subset forms a smaller point cloud, which is processed independently.

During motion, the algorithm calculates the center point of each subset by averaging the X and Y coordinates of all points on the respective side. If one side lacks a sufficient number of points—typically indicating missing maize plants—the center is not computed. In such cases, the center from the opposite side is mirrored across the robot's coordinate axis to generate a virtual point on the missing side. This process requires prior knowledge of maize row spacing, as the correct mirroring is achieved by adding or subtracting the known inter-row distance.

To validate functionality, the algorithm generates markers in Rviz, showing the positions of the computed centers. Figure 6 illustrates this process: the left panel displays the case when valid centers are available on both sides, while the right panel demonstrates a scenario where the right-side data is missing and replaced by a mirrored coordinate. This approach enables continuous navigation even in partially populated fields.

If neither side provides sufficient data to calculate a center point, the algorithm enters a loop to check whether the condition persists across several iterations. Should this situation remain unchanged, the system concludes that the robot has reached the end of the row and initiates the turning procedure.

Driving speed and steering corrections are determined by comparing the Y-coordinate differences of the calculated centers. Based on this comparison, the resulting center falls into one of three categories:

- The center lies within a narrow forward band → no correction required, robot continues

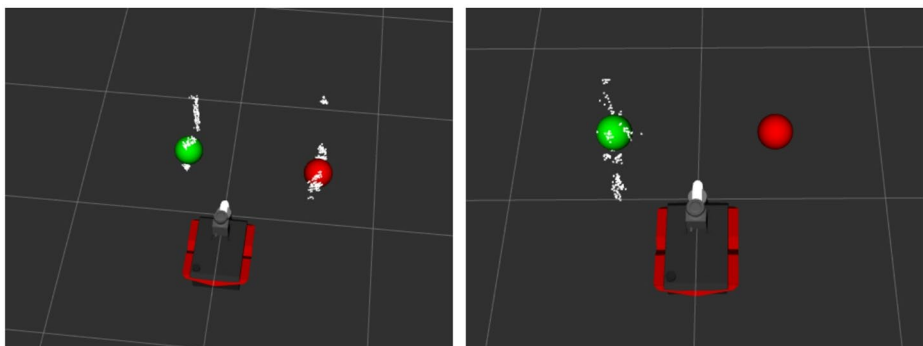


Fig. 6 Computation of navigation center points from LiDAR data. Left: both left and right crop-row point clouds are detected, allowing direct calculation of two center points. Right: when one side is missing, the center point is mirrored across the robot's axis based on known row spacing. The robot moves along the line defined by these calculated centers to maintain a centered trajectory

straight.

- The center lies to the left → robot corrects by steering right.
- The center lies to the right → robot corrects by steering left.

Turning - For successful navigation through the polygon test field, the robot must reliably turn into the next row after completing the current one. In this implementation, turning is based on orientation data from the IMU, represented as quaternions (Zhang et al., 2020). While quaternions provide a robust framework for handling rotations, they are less intuitive than Euler angles, as they use four parameters to represent three degrees of freedom. For certain conditions, such as controlled simulation environments with known orientations, Euler angles may suffice. However, in real-world operation, discontinuities (e.g., jumps from -180° to $+180^\circ$ or 0° to 360°) must be handled, making quaternions the preferred choice.

A quaternion q can be expressed as:

$$q = w + x \cdot i + y \cdot j + z \cdot k \quad (1)$$

$$q = q_w + q_x + q_y + q_z \quad (2)$$

where w represents the real (scalar) part of the quaternion, while $x \cdot i + y \cdot j + z \cdot k$ represents the imaginary (vector) part. The terms i, j, k are unit vectors representing imaginary axes and x, y, z are real numbers indicating their contributions. Equation (2) provides an alternative notation with each component labelled.

The conversion between Euler angles and quaternions is given by:

$$q_w = \cos\left(\frac{\alpha}{2}\right) \quad (3)$$

$$q_x = \sin\left(\frac{\alpha}{2}\right) \cdot \cos(\beta_x) \quad (4)$$

$$q_y = \sin\left(\frac{\alpha}{2}\right) \cdot \cos(\beta_y) \quad (5)$$

$$q_z = \sin\left(\frac{\alpha}{2}\right) \cdot \cos(\beta_z) \quad (6)$$

Here, α represents the rotation angle about the selected axis, while $\beta_x, \beta_y, \beta_z$ represent angles from the rotation axis to each coordinate axis. These β values are expressed as Euler angles, which define the rotation of the body in terms of roll, pitch, and yaw. In this formulation, they are used as intermediate parameters that describe the axis of rotation in Euler space, before being converted into the quaternion representation.

Simplified, if the rotation axis is around the Z-axis, the quaternion simplifies as:

$$q_w = \cos\left(\frac{\alpha}{2}\right) \quad (7)$$

$$q_z = \sin\left(\frac{\alpha}{2}\right) \quad (8)$$

These equations calculate the quaternion representing the desired rotation, typically 90° , -90° or 180° . To perform the turn, the current quaternion is stored at the start of the turn. Unlike Euler angles, where the rotation is simply added, quaternions require multiplication:

$$q_{final} = q_{rotation} \cdot q_{initial} \quad (9)$$

Here, q_{final} represents the target quaternion after the turn, $q_{rotation}$ is calculated using Eqs. (7) and (8), and $q_{initial}$ is the starting orientation.

Finally, the current quaternion is compared to the target quaternion during the turn. Two quaternions are considered equal when their scalar product equals 1:

$$q_{final} \cdot q_{initial} = 1 - \epsilon \quad (10)$$

In Eq. (10), ϵ represents a small permissible deviation. In the code, the algorithm checks when the orientations are sufficiently close, falling within the defined margin of ϵ .

Results and discussion

The FarmBeast robot's autonomous navigation system was evaluated under multiple scenarios to verify the improvements of the new algorithm compared to the legacy version. Key performance indicators—navigation speed, crop safety, and algorithm robustness—were used to assess effectiveness in both simulated and physical environments. Figure 7 illustrates a simulated cornfield created in Gazebo, while Fig. 8 presents the physical test setup with artificial maize rows used for comparative testing.

For each setup, ten trials were conducted with the old algorithm and ten with the new one, across both simulation and real environments. During each trial, navigation time was recorded and the number of maize plants knocked over was counted. Tests were terminated either upon successful traversal of the entire field or when more than five plants were damaged. Only successful trials were included in the calculation of average driving time and crop collisions. Results are summarized in Tables 1, 2 and 5–6 for simulation tests, and Tables 3, 4 and 7–8 for controlled physical experiments with the real robot.

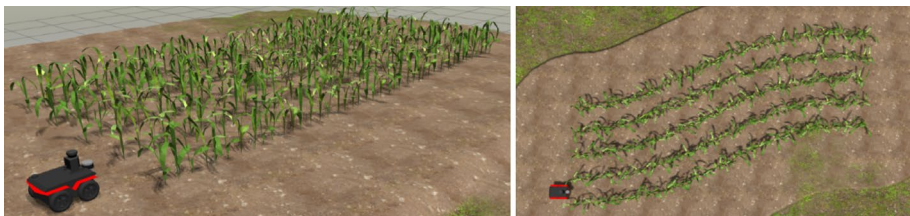


Fig. 7 Gazebo simulation environments used for algorithm development and testing. Left: structured crop rows arranged in straight lines. Right: curved crop rows presenting a more complex navigation challenge. These simulated fields allow controlled evaluation of algorithmic performance under repeatable conditions



Fig. 8 Controlled physical testing environment used for comparative evaluation of navigation algorithms. Artificial maize plants were arranged indoors to replicate crop rows. Left: setup with straight rows. Right: setup with curved rows. This environment allowed repeatable and supervised testing of the FarmBeast robot while avoiding damage to live crops

Table 1 Simulation test results (straight rows) using the legacy navigation algorithm

Test	Traversed all?	Time [s]	Deviation from average time [s]	Knocked over plant	Deviation from average collisions From AVG [#]	Comment
#1	YES	113.94	8.36	0	-1.29	
#2	NO	47.97	X	5+	X	Plant knocked over when turning. Heading not recovered.
#3	YES	101.19	-4.39	1	-0.29	Plant knocked over when turning. Recovered heading.
#4	YES	106.28	0.70	5	3.71	Plant knocked over when turning. Recovered heading.
#5	NO	99.29	X	5+	X	Plant knocked over when turning. Heading not recovered.
#6	YES	106.13	0.55	0	-1.29	
#7	YES	103.30	-2.28	2	0.71	Plant knocked over when turning. Recovered heading.
#8	NO	65.46	X	5+	X	Plant knocked over when turning. Heading not recovered.
#9	YES	104.33	-1.25	1	-0.29	Plant knocked over when turning. Recovered heading.
#10	YES	103.92	-1.66	0	-1.29	
AVG		105.58	4.07	1.29	1.8	

The results presented in Tables 2, 4 and 6, and 8 clearly highlight the superior performance of the new algorithm compared to the outcomes shown in Tables 1, 3 and 5, and 7. While the legacy algorithm often struggled to complete runs—particularly during physical tests—the optimized version successfully completed all trials.

Navigation speed and efficiency

As shown by Fig. 9, the new algorithm demonstrated a marked increase in navigation speed across all environments. In structured crop rows, the algorithm enabled the FarmBeast robot to maintain consistently high performance under regular field geometry. It achieved an average traversal time of 26.94 s per row, representing a 21% reduction compared to the legacy algorithm's average of 32.97 s. This improvement is attributed to optimized filtering and

Table 2 Simulation test results (straight rows) using the optimized navigation algorithm

Test	Traversed all?	Time [s]	Deviation from average time [s]	Knocked over plant	Deviation from average collisions From AVG [#]	Comment
#1	YES	62.96	0.08	0	-0.2	
#2	YES	61.97	-0.91	0	-0.2	
#3	YES	62.99	0.11	0	-0.2	
#4	YES	65.08	2.20	0	-0.2	
#5	YES	62.79	-0.09	0	-0.2	
#6	YES	62.00	-0.88	0	-0.2	
#7	YES	61.35	-1.53	1	0.8	Plant knocked over when turning. Recovered heading.
#8	YES	63.23	0.35	0	-0.2	
#9	YES	62.34	-0.54	0	-0.2	
#10	YES	64.13	1.25	1	0.8	Plant knocked over when turning. Recovered heading.
AVG		62.88	1.09	0.2	0.42	

Table 3 Physical test results (straight rows) using the legacy navigation algorithm

Test	Traversed all?	Time [s]	Deviation from average time [s]	Knocked over plant	Deviation from average collisions From AVG [#]	Comment
#1	NO	16.32	X	0	X	The robot turned in the wrong direction at the end of the row.
#2	NO	15.11	X	3+	X	The robot started turning too early.
#3	NO	16.88	X	0	X	The robot turned in the wrong direction at the end of the row.
#4	YES	32.47	-0.50	1	0.2	
#5	YES	33.09	0.12	0	-0.8	
#6	YES	34.67	1.70	1	0.2	
#7	YES	31.94	-1.03	0	-0.8	
#8	YES	32.70	-0.27	2	1.2	
#9	NO	10.04	X	3+	X	The robot started turning too early.
#10	NO	13.95	X	3+	X	The robot started turning too early.
AVG		32.97	1.04	0.80	0.84	

Table 4 Physical test results (straight rows) using the optimized navigation algorithm

Test	Traversed all?	Time [s]	Deviation from average time [s]	Knocked over plant	Deviation from average collisions From AVG [#]	Comment
#1	YES	23.78	-0.67	0	-0.2	
#2	YES	25.32	0.87	0	-0.2	
#3	YES	24.64	0.19	0	-0.2	
#4	YES	26.31	1.86	0	-0.2	
#5	YES	23.39	-1.06	0	-0.2	
#6	YES	26.28	1.83	0	-0.2	
#7	YES	23.77	-0.68	0	0.8	
#8	YES	22.42	-2.03	1	-0.2	Plant knocked over when turning at row entry. Recovered heading.
#9	YES	24.14	-0.31	0	-0.2	
#10	YES	24.47	0.02	0	0.8	
AVG		24.45	1.24	0.1	0.32	

Table 5 Simulation test results (curved rows) using the legacy navigation algorithm

Test	Traversed all?	Time [s]	Deviation from average time [s]	Knocked over plant	Deviation from average collisions From AVG [#]	Comment
#1	YES	123.68	0.75	4	1.83	Plant knocked over when turning at row entry and within the row. Recovered heading.
#2	YES	125.03	2.1	1	-1.17	Plant knocked over when turning at row exit. Recovered heading.
#3	YES	122.18	-0.75	2	-0.17	Plant knocked over when turning at row exit. Recovered heading.
#4	NO	102.36	X	5+	X	Plant knocked over within the row. Heading not recovered.
#5	YES	125.74	2.81	3	0.83	Plant knocked over within the row. Recovered heading.
#6	YES	92.57	X	5+	X	Plant knocked over within the row. Heading not recovered.
#7	YES	72.69	X	5+	X	Plant knocked over within the row. Heading not recovered.
#8	YES	122.55	-0.38	1	-1.17	Plant knocked over within the row. Recovered heading.
#9	YES	118.40	-4.53	2	-0.17	Plant knocked over within the row. Recovered heading.
#10	NO	87.63	X	5+	X	Plant knocked over when turning at row entry. Heading not recovered.
AVG		122.93	2.61	1.29	1.17	

data processing, which reduced computational overhead while preserving environmental accuracy. In curved row scenarios, the speed gain was even greater, with improvements of up to 35% in traversal time during complex turns.

Crop damage and safety

Crop safety emerged as a key area of improvement. In straight-row tests, the new algorithm reduced plant collisions by 88%, while in curved rows the reduction reached 91%. By contrast, the legacy algorithm frequently caused plant damage, particularly during turning and realignment maneuvers. These results confirm that the new algorithm achieves higher accuracy while preserving crop integrity.

Algorithm robustness and consistency

The robustness of the system was further validated by its consistency across repeated trials, depicted by Fig. 10. In structured row tests, the new algorithm successfully completed all 10 runs, achieving a very low standard deviation of 0.39 s in traversal time. The legacy algorithm, on the other hand, showed only a 60–70% success rate, with considerable variability and frequent failures during row transitions or obstacle encounters.

Simulation vs. Physical testing

A key objective of this work was to ensure strong alignment between simulation and physical testing. Results demonstrated a high correlation between Gazebo/Rviz simulations and

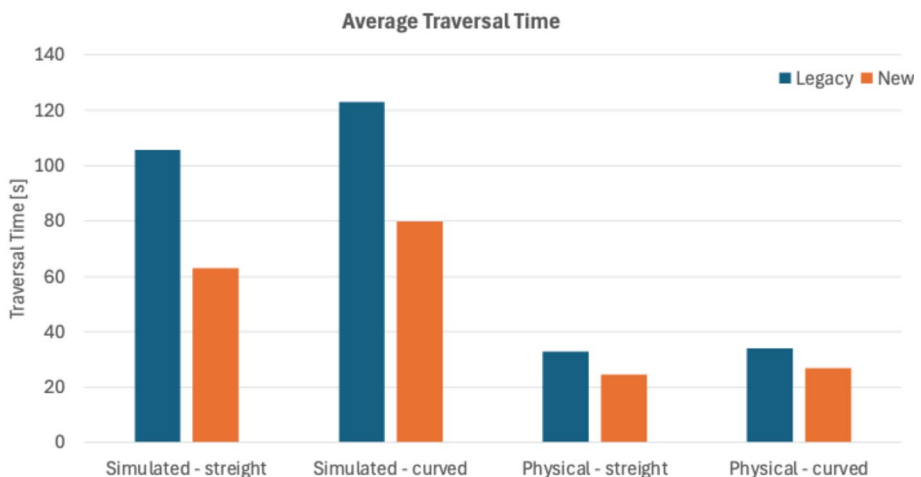


Fig. 9 Average traversal time for straight and curved row layouts in both simulated and physical tests, comparing the legacy and optimized navigation approaches. Values are based on calculated averages from Tables 1, 2, 3, 4, 5, 6 and 7, and 8

physical trials, confirming both the validity of the simulation models and the readiness of the algorithm for deployment. Moreover, the new algorithm was successfully applied in the Field Robot Event 2024, further validating its robustness and performance under competitive field conditions.

Conclusion

Overall, this study contributes by presenting the development and evaluation of an optimized autonomous navigation system for agricultural applications. By comparing the new algorithm with its predecessor across performance metrics such as speed, accuracy, and crop preservation, the findings highlight the potential of autonomous field robots to play a significant role in precision agriculture and sustainable farming practices.

The modular structure of the proposed navigation framework also facilitates the integration of new functionalities for future upgrades. Specifically, the combination of advanced filtering methods, quaternion-based orientation tracking, and state machine control offers a scalable solution that can be extended to more complex agricultural scenarios. These design choices not only improve robustness and reliability but also establish a foundation for incorporating additional sensor modalities, adaptive decision-making, and task-specific behaviors.

The results confirm the hypotheses regarding improvements in navigation speed, accuracy, and crop safety. The enhanced navigation algorithm enables the FarmBeast robot to sustain high performance in terms of speed and precision while minimizing crop damage. In structured crop rows, the algorithm enabled the FarmBeast robot to maintain consistently high performance under regular field geometry. These enhancements led to statistically measurable improvements in traversal speed and reliability, as demonstrated by consistent

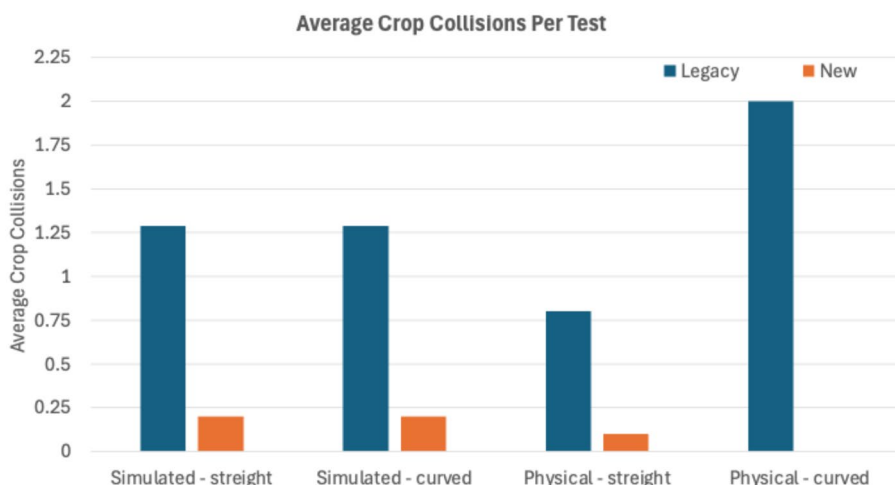


Fig. 10 Number of crop collisions per test for straight and curved row layouts in both simulated and physical environments, comparing the legacy and optimized navigation approaches. Values are based on data from Tables 1, 2, 3, 4, 5, 6 and 7, and 8

completion rates and reduced standard deviation across repeated trials. Overall, the combination of advanced filtering methods and structured state management offers a balanced and effective solution for autonomous navigation in structured agricultural environments.

Looking ahead, further optimization of sensor fusion—particularly during turning manoeuvres—could strengthen robustness by allowing LiDAR and IMU data to jointly validate both orientation and crop presence. Likewise, refining the finite-state machine (FSM) to manage more complex navigation tasks and adapt dynamically to variable field conditions would enhance flexibility, extending the system’s applicability to a wider range of agricultural scenarios.

The optimized navigation system demonstrates significant performance improvements over its predecessor, exceeding the prior system in all key evaluation metrics, including traversal speed, success rate, and crop safety, as shown in Tables 1, 2, 3, 4, 5, 6, 7 and 8; Figs. 9, 11 and 10. These statistically supported results represent a meaningful step toward practical and sustainable automation in real agricultural settings. By achieving a balanced combination of computational efficiency, operational speed, and crop safety, the proposed system strengthens the foundation for long-term adoption of precision farming technologies and autonomous solutions in structured crop environments.

Table 6 Simulation test results (curved rows) using the optimized navigation algorithm

Test	Traversed all?	Time [s]	Deviation from average time [s]	Knocked over plant	Deviation from average collisions From AVG [#]	Comment
#1	YES	82	2.259	0	−0.2	
#2	YES	81.97	2.229	0	−0.2	
#3	YES	89.68	−0.061	0	−0.2	
#4	YES	78.27	−1.471	0	−0.2	
#5	YES	79.53	−0.211	0	−0.2	
#6	YES	78.97	−0.771	0	−0.2	
#7	YES	80.29	0.549	1	0.8	Plant knocked over when turning at row entry. Recovered heading.
#8	YES	78.63	−1.111	0	−0.2	
#9	YES	80.12	0.379	1	0.8	Plant knocked over when turning at row entry. Recovered heading.
#10	YES	77.95	−1.791	0	−0.2	
AVG		79.74	1.40	0.2	0.42	

Table 7 Physical test results (curved rows) using the legacy navigation algorithm

Test	Traversed all?	Time [s]	Deviation from average time [s]	Knocked over plant	Deviation from average collisions From AVG [#]	Comment
#1	YES	35.49	1.53	1	1	
#2	YES	34.03	0.07	0	-2	
#3	YES	34.61	0.65	3	1	
#4	NO	17.5	X	3+	X	Plant knocked over when entering the row. Heading not recovered.
#5	YES	33.71	-0.25	3	1	
#6	YES	32.88	-1.08	3	1	
#7	NO	23.06	X	3	X	Plant knocked over within the row. Heading not recovered.
#8	YES	33.63	-0.33	3+	0	
#9	NO	13.4	X	3+	X	The robot started turning too early.
#10	YES	3.40	-0.56	2	0	
AVG		33.96	0.86	2	1.15	

Table 8 Physical test results (curved rows) using the optimized navigation algorithm

Test	Traversed all?	Time [s]	Deviation from average time [s]	Knocked over plant	Deviation from average collisions From AVG [#]	Comment
#1	YES	26.91	-0.03	0	0	
#2	YES	26.51	-0.43	0	0	
#3	YES	26.92	-0.02	0	0	
#4	YES	27.42	0.48	0	0	
#5	YES	27.44	0.50	0	0	
#6	YES	26.82	-0.12	0	0	
#7	YES	26.64	-0.30	0	0	
#8	YES	26.33	-0.61	0	0	
#9	YES	27.03	0.09	0	0	
#10	YES	27.42	0.48	0	0	
AVG		26.94	0.39	0	0	

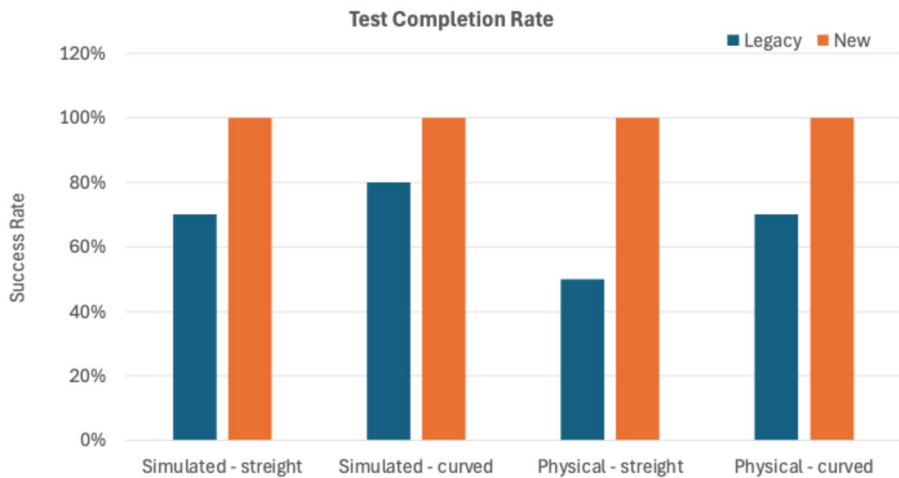


Fig. 11 Test completion rate (%) for straight and curved row layouts in both simulated and physical environments, comparing the legacy and optimized navigation approaches. Completion rates are calculated based on the number of successful trials reported in Tables 1, 2, 3, 4, 5, 6, 7 and 8

Acknowledgements This research was partially supported by the Agri Digital Growth project (Interreg CE200761 grant) and the CRP - Smernice za pametno in precizno kmetijstvo (V2-2426) project, funded by the Slovenian Ministry of Agriculture, Forestry and Food and the Slovenian Research Agency (ARIS).

Author contributions J.R. conceived the study, developed the research concept, designed the methods, provided the necessary tools and domain knowledge, mentored the co-author, and prepared the initial draft of the manuscript. G.P. implemented the proposed ideas, carried out testing and validation, and contributed materials and revisions to the final version of the manuscript. All authors reviewed and approved the final manuscript.

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

Baby, T., Gohil, M. K., & Bhattacharya, B. (2024). Autonomous navigation of 4WIS4WID agricultural field mobile robot using deep reinforcement learning. *arXiv*. <https://doi.org/10.48550/arXiv.2412.18865>

- Bernad, P., Zajc, A., Friš, R., Mlinarič, J., & Rakun, J. (2019). Farmbeast. In H. Floto & H. W. Griepentrog (Eds.), *Proceedings of the 16th Field Robot Event 2018*. University of Hohenheim, Technology in Crop Production.
- Bohren, J. (n.d.) (Ed.). (2024). SMACH: A task-level architecture for rapidly creating complex robot behavior. Retrieved June 20, from <http://wiki.ros.org/smach>
- FarmBeast (2024). FarmBeast: Autonomous agricultural robots for sustainable farming. Retrieved on September 27, 2024, from <http://farmbeast.um.si>
- Field Robot Event (2023). Field Robot Event 2023: Overview and results. Retrieved September 27, 2024, from <https://fieldrobot.nl/event/>
- Fischler, M. A., & Bolles, R. D. (1981). Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6), 381–395. <https://doi.org/10.1145/358198.358203>
- Li, H., Huang, K., Sun, Y., Lei, X., Yuan, Q., Zhang, J., & Lv, X. (2025). An autonomous navigation method for orchard mobile robots based on 3D point cloud optimization. *Frontiers in Plant Science*, 14, Article 1510683. <https://doi.org/10.3389/fpls.2024.1510683>
- Lv, M., Wei, H., Fu, X., Wang, W., & Zhou, D. (2022). A loosely coupled extended Kalman filter algorithm for agricultural scene-based multi-sensor fusion. *Frontiers in Plant Science*, 13, 849260. <https://doi.org/10.3389/fpls.2022.849260>
- Open Robotics. (n.d.). Gazebo: A robot simulation environment. Retrieved June 17 (2024). from <http://wiki.ros.org/gazebo>
- Panda, S. K., Lee, Y., & Jawed, M. K. (2023). Agronav: Autonomous navigation framework for agricultural robots and vehicles using semantic segmentation and semantic line detection. In *Proceedings of the CVPR 2023 Workshop on Agriculture Vision*. Retrieved from https://openaccess.thecvf.com/content/CVPR2023W/AgriVision/papers/Panda_Agronav_Autonomous_Navigation_Framework_for_Agricultural_Robots_and_Vehicles_Using_CVPRW_2023_paper.pdf
- Popič, G. (2021). *The development of an algorithm for autonomous robot driving in ROS/GAZEBO environment [Razvoj Algoritma Za Avtonomno vožnjo Robota v ROS/GAZEBO okolju: Diplomsko delo]*. University of Maribor, Faculty of Mechanical Engineering.
- Popič, G., Naveršnik, U., Kokalj, R., Rihter, J. J., E., & Rakun, J. (2024). in press, An autonomous field robot FARMBEAST – the Field Robot Event 2023 edition. *Agricultura Scientia*, vol. 21, issue 2.
- Raj, R., Singh, A., & Kumar, P. (2024). A brief overview of technologies in automated agriculture. *International Journal of Environmental and Climate Change*, 14(7), 181–209. <https://doi.org/10.9734/ijeccl/2024/v14i730123>
- Rakun, J. (2015). *Proceedings of the 13th Field Robot Event 2015, Maribor, Slovenia, 16–18 June*. Faculty of Agriculture and Life Sciences, Department of Biosystems Engineering.
- Rakun, J., Pantano, M., Lepej, P., & Lakota, M. (2022). Sensor fusion-based approach for the field robot localization on rovitis 4.0 vineyard robot. *International Journal of Agricultural and Biological Engineering*, 15(6), 91–95. <https://doi.org/10.25165/j.ijabe.20221506.6415>
- ROS.org About ROS. Available at: <https://www.ros.org/about-ros/>
- Safe Software. (n.d.). FME transformers: Voxel grid. Retrieved July 22 (2024). from https://docs.safe.com/fme/html/FME-Form-Dokumentation/FME-Transformers/!FME_Geometry/VoxelGrid.htm
- Velodyne, I. (2015). Velodyne VLP-16 LiDAR sensor: Product specifications. Retrieved September 27, 2024, from <https://www.velodynelidar.com>
- Willmann, J., & Krajnik, T. (2015). Rviz: A 3D visualization tool for ROS. Retrieved June 17, 2024, from <http://wiki.ros.org/rviz>
- Xsens Technologies (2015). MTi-610 inertial measurement unit: Product specifications. Retrieved September 27, 2024, from https://www.xsens.com/hubfs/Downloads/Leaflets/MTi_600-series_Datasheet.pdf
- Yahaya, Z. A., Buyamin, S., Chiroma, H., Azimi Mahmud, M. S., Hassan, F., & Badi, A. A. H. (2022). *Paths planning for agricultural robots: Recent development, taxonomy, challenges, and opportunities for future research*. In *Proceedings of the 2022 IEEE 20th Student Conference on Research and Development (SCORED)* (pp. 7–12). IEEE. <https://doi.org/10.1109/SCORED57082.2022.9973836>
- Zhang, Y., Liu, Y., & Wang, Y. (2020). A new quaternion-based Kalman filter for human body motion tracking using the second estimator of the optimal quaternion algorithm and the joint angle constraint method with inertial and magnetic sensors. *Sensors*, 20(20), Article 5887. <https://doi.org/10.3390/s20205887>